In [ ]:
```
PAWAR SAKSHI MAHENDRA
ROLL NO - 49
PRACTICAL NO - 07
```

In [1]:
```python
import numpy as np

class XORNetwork:
    def __init__(self):
        # Initialize weights and biases randomly
        self.W1 = np.random.randn(2, 2)
        self.b1 = np.random.randn(1, 2)
        self.W2 = np.random.randn(2, 1)
        self.b2 = np.random.randn(1, 1)

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def sigmoid_derivative(self, x):
        return x * (1 - x)

    def forward(self, X):
        # Forward pass
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.sigmoid(self.z1)

        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = self.sigmoid(self.z2)

        return self.a2

    def backward(self, X, y, output, lr):
        # Backward pass
        output_error = y - output
        output_delta = output_error * self.sigmoid_derivative(output)

        z1_error = output_delta.dot(self.W2.T)
        z1_delta = z1_error * self.sigmoid_derivative(self.a1)

        # Update weights with learning rate
        self.W2 += self.a1.T.dot(output_delta) * lr
        self.b2 += np.sum(output_delta, axis=0, keepdims=True) * lr

        self.W1 += X.T.dot(z1_delta) * lr
        self.b1 += np.sum(z1_delta, axis=0, keepdims=True) * lr

    def train(self, X, y, epochs, lr=0.1):
        for i in range(epochs):
            output = self.forward(X)
            self.backward(X, y, output, lr)

            if i % 1000 == 0:
                error = np.mean(np.abs(y - output))
                print(f"Epoch {i}, Error: {error}")
```

```
    def predict(self, X):
        output = self.forward(X)
        return (output >= 0.5).astype(int)


# XOR dataset
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([[0],
              [1],
              [1],
              [0]])

# Create network
xor_nn = XORNetwork()

# Train
xor_nn.train(X, y, epochs=10000, lr=0.1)

# Predict
predictions = xor_nn.predict(X)

print("\nFinal Predictions:")
print(predictions)
```

```
Epoch 0, Error: 0.49556001297880425
Epoch 1000, Error: 0.4511840491136577
Epoch 2000, Error: 0.3960612125535148
Epoch 3000, Error: 0.37442131533507794
Epoch 4000, Error: 0.3645113290234004
Epoch 5000, Error: 0.35885521057073866
Epoch 6000, Error: 0.3551783724119791
Epoch 7000, Error: 0.3525822908387447
Epoch 8000, Error: 0.35064224797413857
Epoch 9000, Error: 0.349131382889748

Final Predictions:
[[0]
 [1]
 [1]
 [1]]
```

In [ ]: